Online Marketplaces

Part 4: Multi Armed Bandits

Bar Light

February 4, 2026

This Lecture: Multi-Armed Bandits (MAB)

1. **Introduction to MAB and Application Domains**
   - The MAB problem: balancing **exploration vs. exploitation**.
   - Applications in **online marketplaces**.
2. **Key Algorithms for MAB**
   - **Epsilon-Greedy** (Simplicity),
     Upper Confidence Bound (UCB): (Frequentist),
     Thompson Sampling (TS): (Bayesian approach).
3. **Challenges in MAB and Solutions**
   - Non-stationarities, short horizons. competition.
   - **Solutions**:
     - **Discounting**, **Sliding Window**
     - Competitive environments.

1. Multi Armed Bandits

- In the multi-armed bandit (MAB) problem, imagine a gambler at a row of slot machines ("arms").

  - Each with its own unknown reward system.

  - Each pull of a slot machine's lever can be seen as a separate trial.

  - The gambler's goal is to develop a strategy to maximize their winnings over a series of plays.

  These problems were extensively studied in Operations, Statistics, Computer Science, and Economics.

Multi Armed Bandits: Exploration and Exploitation

The central dilemma in MAB is the trade-off between "exploration" and "exploitation".

"Exploration" refers to the act of trying different machines to gather information about their potential rewards.

"Exploitation", on the other hand, involves sticking with a machine that has yielded good rewards in the past.

Balancing between exploration and exploitation is the key for developing successful algorithms.

How would you balance?

MAB in Recommendations

Multi-Armed Bandit in Recommendations

- **Platforms**: Netflix, Amazon, Spotify, etc.
- **Purpose**: Improve user engagement / experience by dynamically learning user preferences.
- **Mechanism**:
  - Arms: Represent individual recommendations (e.g., a movie, product, or song).
  - Reward: Metrics like clicks, views, or playback time.
- **Example**: Netflix uses MAB to recommend movies.

Traditional Methods for Recommendations

Collaborative Filtering (CF):

- **User-Based CF**: Recommends items based on preferences of similar users.

  E.g., if User A and User B have rated movies similarly, we recommend movies User A liked to User B.

- **Item-Based CF**: Recommends items similar to those already interacted with.

  E.g., if users who liked Movie X also liked Movie Y, then Movie Y is recommended to users who liked Movie X.

- **Limitations**:
  - Struggles with sparse data.
  - Struggles to recommend for new users and new items (**cold-start problem**).
  - Real-time adaptation is challenging.

Netflix Prize

Netflix Prize (2006–2009): A \$1 million competition to improve Netflix's collaborative filtering algorithm.

Goal: Predict user ratings for movies better than Netflix's Cinematch algorithm.

Training Data: Netflix provided 100M+ ratings from 480K users on 17K+ movies. Each rating included (user, movie, date, rating [1-5]).

Qualifying Set: Netflix provided 2.8M (user, movie, date) tuples. But the actual ratings were hidden.

1M won by a team of researchers: They achieved slightly above 10 percent improvement – the required threshold.

Algorithm never implemented in practice.

Traditional Methods for Recommendations

Matrix Factorization (MF): The user-item interaction matrix is a large table where rows represent users, columns represent items (e.g., movies), and each cell contains a value indicating the user's interaction with that item (e.g., a rating or click). Many cells are typically empty (sparse data).

- Matrix Factorization technique infers missing values in data.
- Handles sparsity better than traditional Collaborative Filtering by breaking down the large matrix into lower-dimensional representations.

Example: If a user has rated several action movies highly, MF learns their preference and can recommend a new action movie, even if they haven't rated it before.

Limitations:

- Computationally expensive.
- Cold-start problem.
- Real-time updates are challenging.

Advantages of MAB for Recommendations

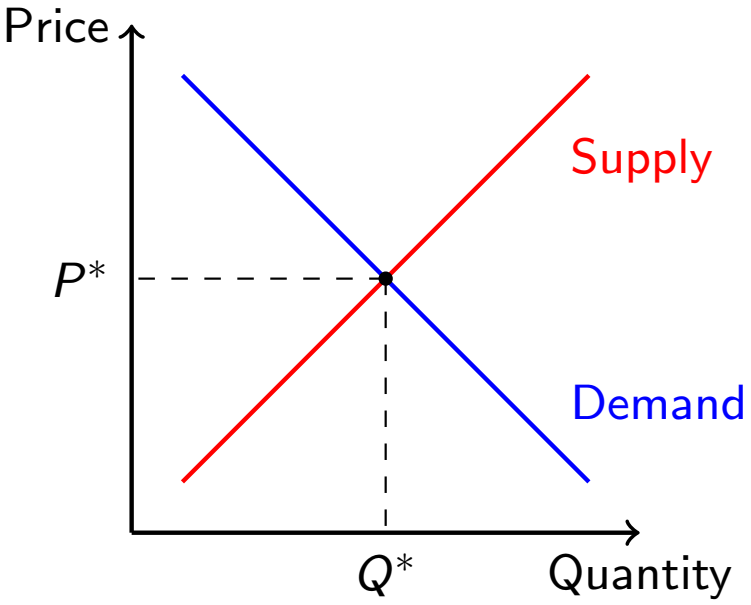
Benefits for Online Platforms:

- **Cold-Start Adaptation**:
  - Effectively recommends without prior data by treating recommendations as "arms" to explore.
- **Contextual Personalization**:
  - Contextual MAB incorporates user/item metadata (e.g., age, location) for personalized recommendations.
- **Integration with Deep Learning**:
  - **Deep MAB models** combine MAB with deep neural networks (e.g., embeddings for user-item interactions).
- **Exploration vs. Exploitation**: CF and MF lack mechanisms to balance exploring new options with leveraging known "good" options.

MAB in Dynamic Pricing

Multi-Armed Bandit in Pricing

- **Platforms**: Uber, Grab, e-commerce platforms.
- **Purpose**: Optimize pricing to maximize revenue and match supply with demand.
- **Mechanism**:
  - Arms: ?
  - Reward: ?
- **Example**: Uber and Grab adjust prices to balance supply and demand.

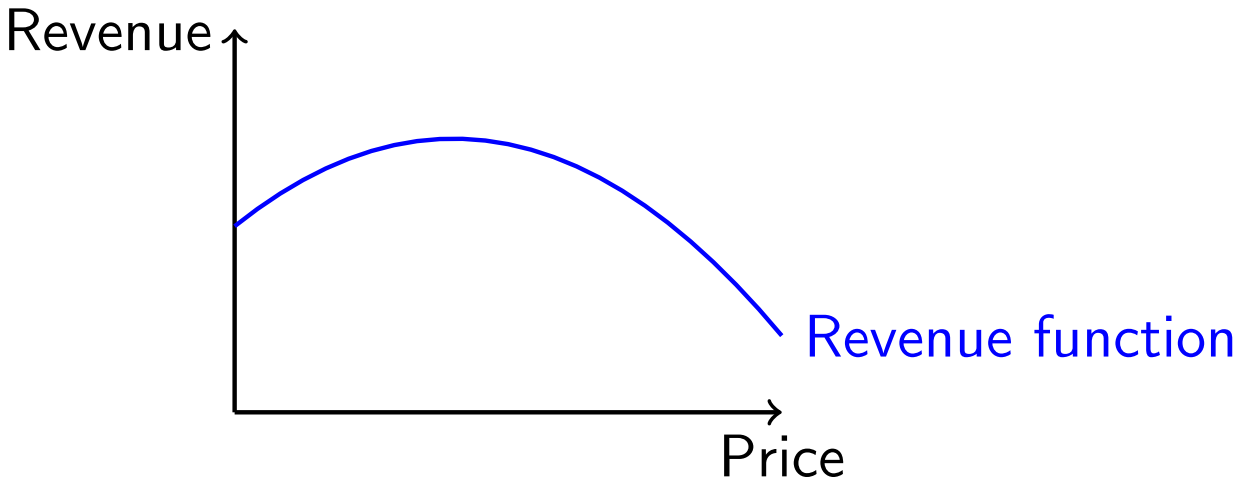


MAB in Dynamic Pricing: Retailer Example

- Suppose that a seller in an e-commerce platform wants to sell a product over a horizon $t = 1, \ldots, T$.
- In modern markets sellers can use complicated dynamic pricing algorithms.
- A typical goal is to maximize total revenue

$$p_1 D(p_1) + \ldots + p_T D(p_T)$$

which depends on the unknown demand function $D(p)$.

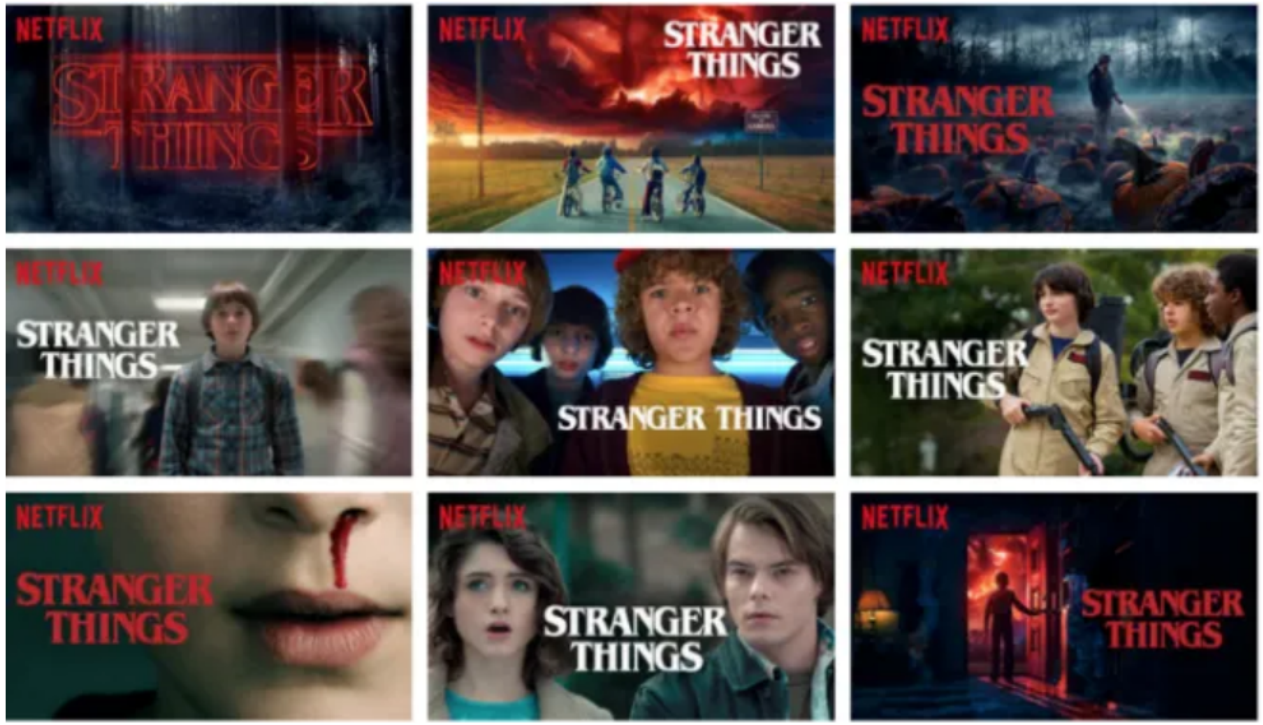


MAB in Experimentation

Multi-Armed Bandit in Experimentation

- **Platforms**: News websites, e-commerce platforms, most platforms.
- **Purpose**: Optimize layouts, ads, feature releases (without the rigid structure of A/B testing).
- **Mechanism**:
  - Arms: Different variations of content (e.g., article layout, ad placement).
  - Reward: Click-through rate (CTR), other?
- **Example**: A news website uses MAB to experiment with headlines and article placements, optimizing CTR based on user interactions.

Netflix Experimentation



Netflix needs to select the best artwork (thumbnail) for a new series, e.g., "Stranger Things" to maximize user engagement.

MAB in Ad Auctions

Multi-Armed Bandit in Ad Auctions

- **Platforms**: Google Ads, Facebook Ads.
- **Purpose**: Maximize advertiser revenues by dynamically adjusting bidding strategies (auto-bidding).
- **Mechanism**:
  - Arms: Different bids.
  - Reward: Clicks, conversions, etc.
  - Example: Google Ads provides auto-bidding services for their advertisers.

General Modeling Approaches

- Three main approaches to tackle MAB problems: Bayesian, Frequentist, and Adversarial.
- Key difference lies in how each approach deals with uncertainty about the rewards or unknowns.

How to model uncertainty?

- The Bayesian approach assumes a prior belief over rewards. The decision maker selects actions based on beliefs and (Bayes) updates after observations.
- The frequentist approach assumes that a true but unknown reward distribution exists.
- The adversarial approach assumes rewards can be strategically influenced by an external force. Models a setting where a competitor reacts adversarially to our actions.

MAB: Frequentist Approach - No Features

- There is a set of arms $A = \{1, \ldots, K\}$.
- Each arm $i \in A$ yields a random reward that is represented by a "true" random variable $r_{i,t}$ that has an average reward μ_i.
- There are T periods. At each time $t = 1, 2, \ldots, T$, an agent chooses an arm $i \in A$ and receives a (random) reward $r_{i,t}$.
  What is an algorithm (for MAB)?

MAB Example: Bernoulli Arms

Setup:

- **Arms**: $A = \{1, 2, 3\}$, representing 3 different ad designs for an online campaign.
- **Rewards**: Each arm yields a binary reward:
  - Success ($r_{i,t} = 1$): User clicks the ad.
  - Failure ($r_{i,t} = 0$): User ignores the ad.
- **True Click-Through Rates (CTR)**:
  - Arm 1: $\mu_1 = 0.30$ (30% success rate).
  - Arm 2: $\mu_2 = 0.50$ (50% success rate).
  - Arm 3: $\mu_3 = 0.40$ (40% success rate).

The Trade-Off: Exploration vs. Exploitation

Exploration:

- **What**: Test arms to discover the true CTR of each arm.
- **Cost**: May spend time on low-performing arms.
- **Example**: Testing Arm 1 ($\mu_1 = 0.30$) even though Arm 2 ($\mu_2 = 0.50$) appears better.

Exploitation:

- **What**: Focus on the arm believed to have the best CTR.
- **Cost**: Risk of missing better arms due to insufficient testing.

Simple way to balance exploration and exploitation...?

Epsilon-Greedy Algorithm

- Epsilon-Greedy is one of the simplest algorithms for MAB.
- At each step, the algorithm decides:
  - **With probability ϵ (Explore):**
    - Randomly select any arm (option) to test.
    - Purpose: Gather more data about less-played arms.
  - **With probability $1 - \epsilon$ (Exploit):**
    - Choose the arm with the highest estimated mean reward (greedy).
    - Purpose: Maximize immediate rewards.

Learning:

- After each step we observe the reward and update the estimated mean.

Epsilon-Greedy Algorithm: Formal

Algorithm Epsilon-Greedy

Input: Exploration rate ϵ_t
for all arm $a \in \{1, \ldots, K\}$ **do**
    Play arm a, observe $r_{a,1}$. Initialize $\hat{\mu}_a = r_{a,1}$, $N_a = 1$
end for
for $t = K + 1$ to T **do**
    Generate $p \sim \text{Uniform}(0, 1)$
    If $p < \epsilon_t$: Select random arm a_t, **else**: Select $a_t = \arg\max_a \hat{\mu}_a$
    Play a_t, observe reward $r_{a_t,t}$
    Update $\hat{\mu}_{a_t} = \frac{\hat{\mu}_{a_t} \cdot N_{a_t} + r_{a_t,t}}{N_{a_t}+1}$, $N_{a_t} = N_{a_t} + 1$
end for

Epsilon-Greedy Algorithm: Step-by-Step Example

Example: Three arms (**A**, **B**, **C**), with unknown rewards (0 or 1).

Step 1: Initialization (Each arm is played once)

Arm	Observed Reward	Estimated Reward	Times Played (N_a)
A	1	1.00	1
B	0	0.00	1
C	1	1.00	1

Step 2: Next Action (Using ϵ-Greedy)

- Generate $p \sim \text{Uniform}(0, 1)$. If $p < \epsilon$, **explore**, otherwise, **exploit**.
- Assume we play **Arm A**, observe reward $= 0$.

Step 3: Update Estimates

Arm	Updated Estimated Reward ($\hat{\mu}_a$)	Times Played (N_a)
A	$\frac{(1\times1)+0}{2} = 0.5$	2
B	0.00	1
C	1.00	1

(Then repeat Steps 2 and 3)

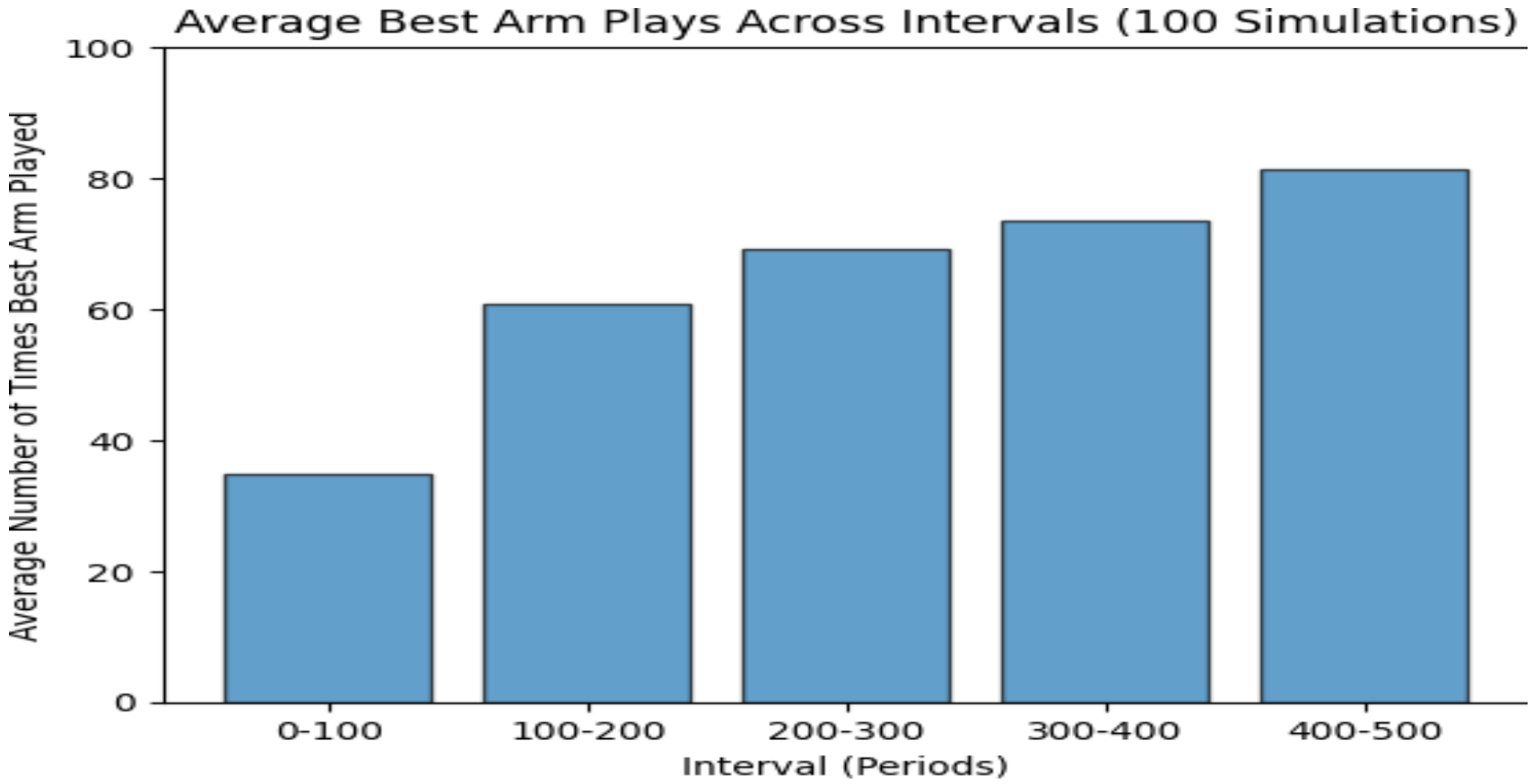
Class Exercise: Epsilon-Greedy

In class: Implement the epsilon-greedy algorithm for 3 Bernoulli arms with success rates of 0.3, 0.4, and 0.5 over 500 periods and a fixed $\epsilon = 0.1$.

Calculate the percentage of times the best arm is chosen in intervals of 0–100, 100–200, 200–300, and so on.

Epsilon-Greedy

Variance of a single simulation is big: the outcome depends heavily on randomness.
Running the algorithm 100 times and averaging yields:



How Do We Measure if an Algorithm is Good?

Goal: We want to evaluate how well a multi-armed bandit algorithm performs over time.

More generally: how should we evaluate algorithms?

- Does it **find the best arm quickly**?
- Does it **maximize total reward**?
- Can we measure **how much we lose** due to exploring suboptimal arms?

Regret

Regret measures how much reward is lost on average by not always selecting the best arm.

- Let r^* be the average reward of the best arm.
- Let r_{a_t} be the average reward obtained by our algorithm at time t.

Regret Definition:

$$R(T) = Tr^* - \sum_{t=1}^{T} r_{a_t}$$

Intuition: Quantifies the (average) regret incurred by the algorithm by not knowing which arm is best.

Regret in the Epsilon-Greedy Example

- Best arm has an average reward of $r^* = 0.5$.
- The algorithm selects different arms over time.

Regret:

$$R(T) = 0.5T - \sum_{t=1}^{T} r_{a_t}$$

Observations:

- If the algorithm (always) plays the best arm, regret would be on average 0.
- A good algorithm should keep regret **as low as possible**.

No-Regret Algorithms

An algorithm has **no regret** if, as we play more rounds and T is large, then $R(T)/T$ becomes very small (close to 0).

- Early on, we try different arms, so regret is large.
- Later, good algorithms mostly pick the best arm, so the regret becomes small.
- How fast $R(T)/T$ becomes small?
- What about more ambitious $R(T)/f(T)$ for "sublinear" f (e.g., $T^{0.5}$ or $\log(T)$)?

Next: UCB and Thompson Sampling algorithms, which are no-regret and optimal in the regret rate.

Optimism in the Face of Uncertainty

- Another simple idea for MAB algorithm is to try to construct **confidence intervals** around the sample mean of every arm and use it to make decisions.

  But which arm should we play at each round?
- Key Idea – optimism in the face of uncertainty. Assume the best about unknowns: encourages playing less explored actions.

Example

- A new restaurant joins GrabFood but has only a few reviews.
- Standard ML design may recommend mainly highly-rated, well-known restaurants.
- Optimism in the face of uncertainty mitigates cold start problem.

The Upper Confidence Bound (UCB) Algorithm

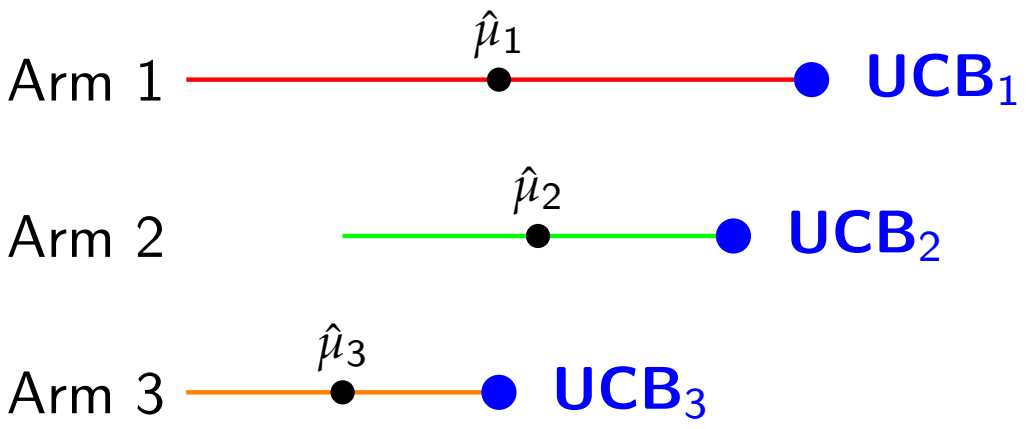
- UCB implements optimism in the face of uncertainty.
- At each step, UCB constructs a confidence interval on the sample mean and and chooses the action with the highest upper confidence bound.

UCB Idea

- The algorithm calculates an **upper confidence bound** for each arm:

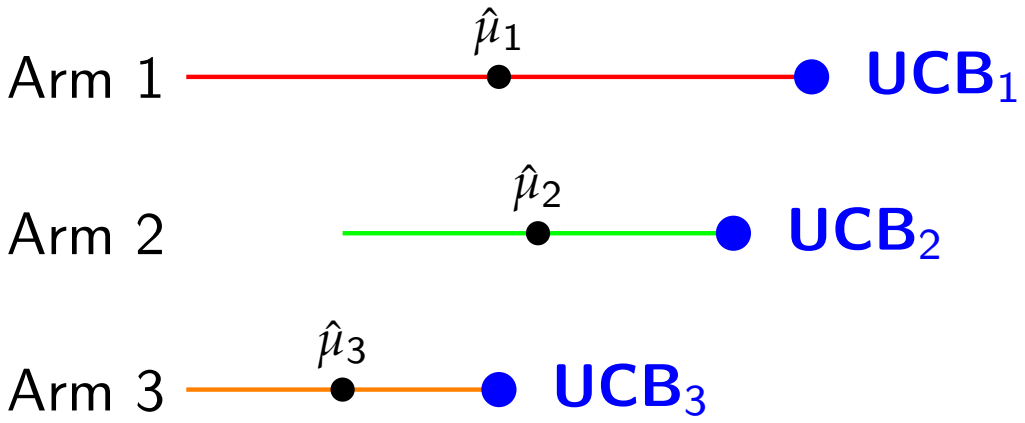
$$\text{UCB}_i = \hat{\mu}_i + \text{bonus factor}$$

  - $\hat{\mu}_i$: Estimated mean reward (based on observed data).
  - **Bonus Factor**: Reflects how uncertain we are (more uncertainty = larger bonus).

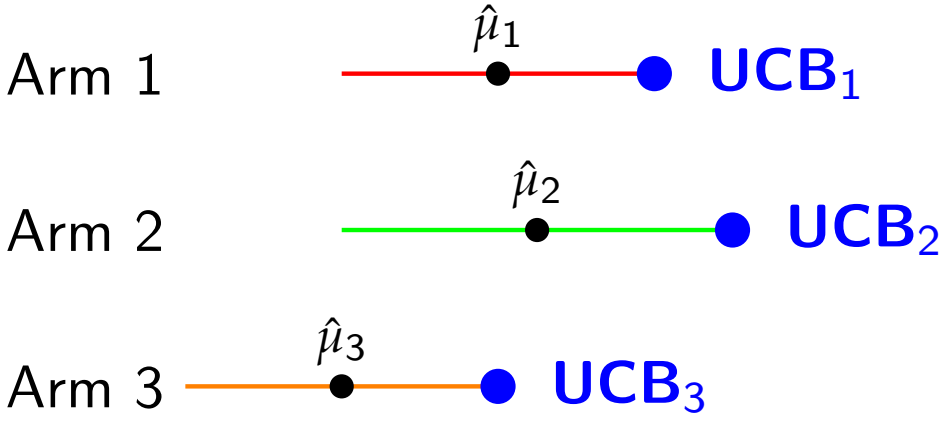


Arm 1 is chosen despite lower estimated mean compared to Arm 2.

UCB Over Time



After playing Arm 1, uncertainty is reduced.



UCB Balancing

How UCB Balances Exploration and Exploitation:

- The upper confidence bound can be seen as a mix of exploitation (the estimated reward) and an exploration bonus (size of the confidence interval).
- It favors arms with high estimated rewards but also prioritizes underexplored arms.

UCB Algorithm: Formal Description

Algorithm UCB1 Algorithm

```
for all arms a do
    Play arm a once, observe reward X_{a,1}
    Set N_a = 1, S_a = X_{a,1}
end for
for t = K + 1, ..., T do
    Select arm a_t = arg max_a [ S_a/N_a + sqrt( 2 log t / N_a ) ]
    Play arm a_t, observe reward X_{a_t,t}
    Update counts: N_{a_t} ← N_{a_t} + 1
    Update cumulative rewards: S_{a_t} ← S_{a_t} + X_{a_t,t}
end for
```

There are many variants of the UCB algorithm but the general idea is the same: select the action with the highest upper confidence bound to balance exploration and exploitation.

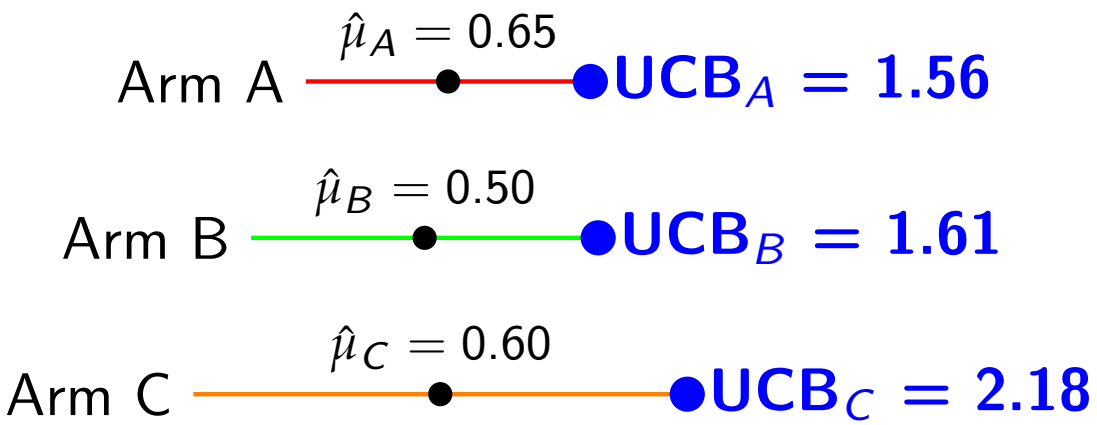
UCB Algorithm: Step-by-Step Example

Example: Three arms (A, B, C) at time $t = 12$.

Arm	Estimated Reward ($\hat{\mu}_a$)	Times Played (N_a)	UCB Value
A	0.65	6	$0.65 + \sqrt{\frac{2 \log 12}{6}}$
B	0.50	4	$0.50 + \sqrt{\frac{2 \log 12}{4}}$
C	0.6	2	$0.6 + \sqrt{\frac{2 \log 12}{2}}$

Step 1: Select the Arm with the Highest UCB

- Compute $\text{UCB}_a = \hat{\mu}_a + \sqrt{\frac{2 \log t}{N_a}}$ for each arm.

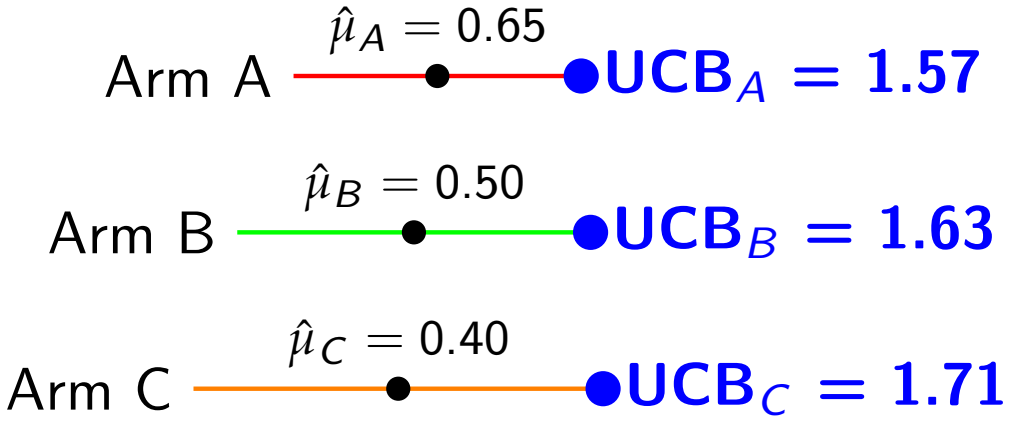


UCB Algorithm at Time $t = 13$ After Playing Arm C

Updated Estimates at $t = 13$. Suppose Arm C gave a reward of 0.

Arm	**Estimated Reward** $(\hat{\mu}_a)$	**Times Played** (N_a)	**UCB Value**
A	0.65	6	$0.65 + \sqrt{\frac{2 \log 13}{6}}$
B	0.50	4	$0.50 + \sqrt{\frac{2 \log 13}{4}}$
C	0.40	3	$0.40 + \sqrt{\frac{2 \log 13}{3}}$

Updated Confidence Intervals and UCB Values:



Arm C is played next again. UCB_A and UCB_B increased.

Bonus Factor in UCB

Why this formula?

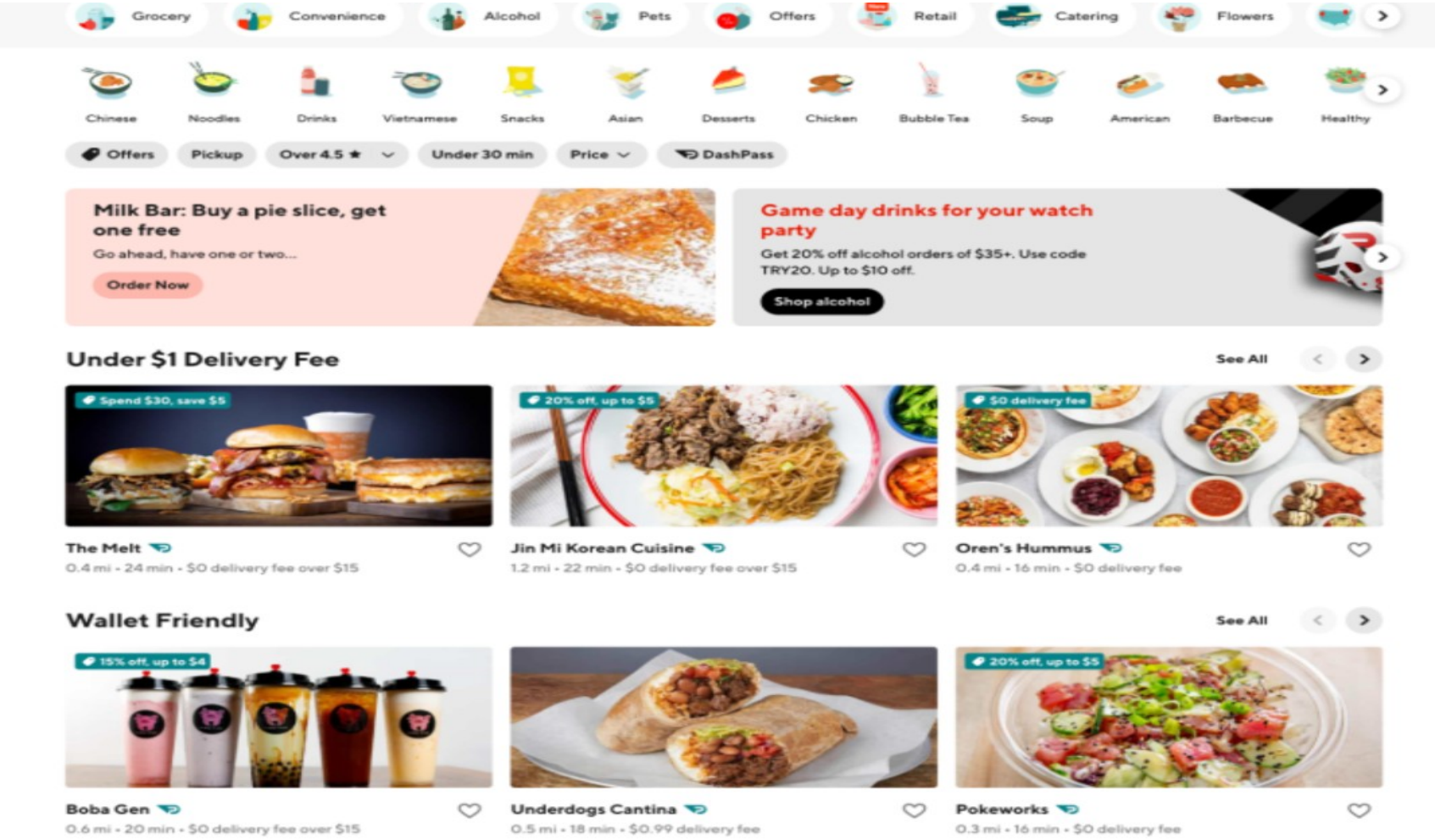
- The bonus factor in UCB is given by:

$$\sqrt{\frac{2 \log t}{N_a}}$$

- This is **not the only possible choice**.
- Intuitively the bonus term represents the "correct" uncertainty in the estimate of each arm's mean in a **frequentist** setting.

Why it can go wrong?

Feed Construction



UCB Scores in Practice: Uber Eats Example

Uber Engineering Blog: Recommending for the Marketplace

Data Science Problem (Uber Eats):

- Rank restaurants on the home screen so eaters discover places they want (hard problem: real estate dilemma..huge candidates list.)
- **The Old Way (Greedy):** Optimized mainly for **conversion rate**.
- But Uber Eats is a **three-sided marketplace** (eaters, restaurants, delivery partners).
- A "Greedy" strategy can hurt the ecosystem:
  - Cold start problem.
  - Experience / long-run outcomes depend on operational signals (e.g., prep/pickup/delivery-time related signals).

Why UCB? Exploration for New Restaurants

- Uber uses **UCB** as an exploration mechanism: under-exposed restaurants get a **boost**.
- As impressions accumulate, uncertainty decreases and the boost naturally fades.
- Uber constructs a **UCB score** for each restaurant based on:
  - **Impressions** (how often the restaurant was shown),
  - **Clicks** (how often users engaged),
  - And a **boosting factor** that encourages exploration under uncertainty. How to tune it?
- That UCB score is then incorporated into a **multi-objective ranking** system (relevance + diversity + long-term satisfaction + marketplace fairness).
- Different policy from UCB algorithm. Here the UCB score acts as a **feature** in a massive **optimization problem** to determine the ranking.

DoorDash Homepage (UCB Example)

▸ DoorDash Engineering Blog: Homepage Recommendation with Exploitation and Exploration

Data Science Problem (Feed Construction):

- DoorDash homepage has limited real estate. Need to decide what to show (and in what order).
- Classic challenge: **feedback loop**
  - Showing an entity more $\Rightarrow$ it gets more clicks/orders $\Rightarrow$ ranker believes it is better $\Rightarrow$ it is shown even more ("rich get richer").

DoorDash Approach: Retrieval → Ranking → Page (UR + UCB)

High-level pipeline:

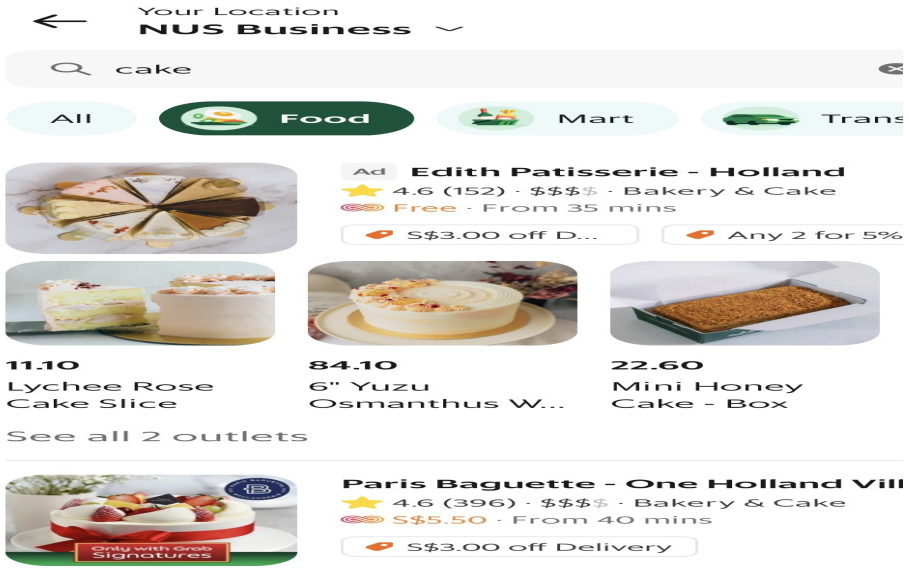
- Step 1: **Candidate generation / retrieval**
  - Collect a set of candidate entities that could be shown on the homepage (reduce potential candidate list).
- Step 2: **Universal Ranker (UR)** for exploitation
  - A deep learning model predicts **pConv** (probability of conversion) for a consumer-entity pair.
- Step 3: **Add exploration using UCB-style uncertainty**
  - Estimate uncertainty using **effective impressions** (the entity was actually shown to the consumer) for that consumer–entity pair.

The Policy: Composite Score (Exploitation + Exploration) and Ranking

What the system does at decision time:

- For each candidate entity: use **UR score** (predicted pConv). and uncertainty / exploration bonus. Combine them into a **composite score**.
- **Rank** entities by that score (compare to UCB?)
- The exploration strength is controlled by an **exploration coefficient** that is tuned.
- Takeaway:
  - Improves personalization while still giving **new/under-exposed** entities a chance.
  - Helps mitigate feedback loops.

Assortment Optimization Reminder



- Data Scientist 1: Believes NUS students are highly successful and prefer **highest quality and price**.
- Data Scientist 2: Thinks NUS students are too busy and prioritize **short delivery times**.
- Last lecture they decide to **run an A/B test**.

Data Scientist 3 (fresh graduate from NUS) suggests using MAB algorithm, which can learn the best assortment **without prior knowledge** of NUS students' preferences.

Class Exercise: Assortment Optimization with UCB

Problem:

- 4 restaurants so we have $4! = 24$ restaurant assortments.
- Purchase probabilities are unknown, for simplicity, assume random in $[0.3, 0.7]$.
- Goal: Find good assortments with UCB.

Implement the UCB algorithm with $T = 10,000$.
Plot how often each assortment is chosen.

How would you compare the A/B testing to the MAB approach?

Why this approach would not make sense in practice (compare with Uber's blog)?

MAB compared to A/B Testing and offline ML

Advantages:

- **Suboptimal choices:** A/B testing can allocate many trials to a poor option, while MAB quickly shifts to better choices.
- **Continuous adaptation:** Unlike A/B testing, MAB does not require predefined experiment periods.
- **Testing multiple options:** MAB allows testing all 24 assortments instead of just two, making it more efficient for multi-option decisions.

But A/B testing provides clear causal inference and easy to implement.

Compared to **offline ML methods** (based on previous purchases).
MAB provides easier integration of new restaurants / other changes.

We can (and should!) use ML (how?)

Stitch Fix: Bandits as an Alternative to A/B Testing

▸ Stitch Fix Tech Blog: Multi-Armed Bandits and the Experimentation Platform

Stitch Fix is an online personal styling service that uses data + algorithms to recommend and ship clothing selections to customers.

Stitch Fix design goal:

- **#oneway**: one company-wide system to run and analyze experiments (used by engineers, PMs, and data scientists). Why?
- Experiments are defined using two building blocks:
  - **Configuration parameters** = what is being varied (UI text, algorithm version, ML hyperparameter, etc.).
  - **Randomization units** = what entity you randomize on (e.g., visitor_id vs stylist_id).
- Built **bandit experimentation** into their existing experimentation platform (same product/engineering interface, bandit logic behind the scenes).

Bayesian Algorithms: Thompson Sampling

- Thompson Sampling is a Bayesian method: it maintains a probability distribution over the reward parameters for each arm.
- At each step, it samples from these distributions to select an arm.
- Observed rewards are used to update the distributions.

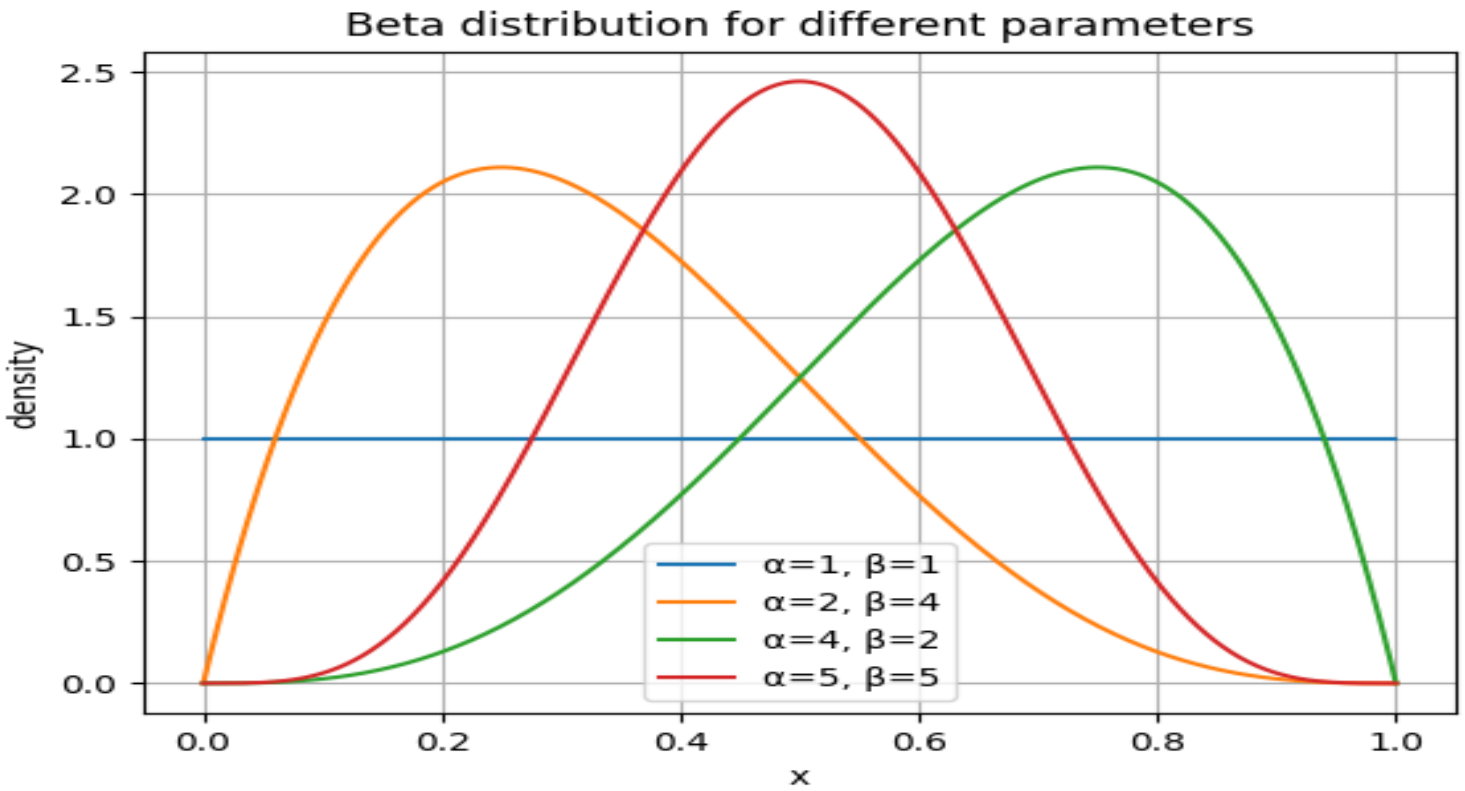
TS: Beta-Bernoulli Model

- Let's consider the case where rewards are binary (0 or 1). and arm i yields a payoff of 1 w.p. μ_i and 0 otherwise.
- Prior is a Beta distribution: $B(\alpha, \beta)$. Determined by two shape parameters, α and β.
- Initially, we might set $\alpha = 1$ and $\beta = 1$ (which corresponds to the uniform distribution) for all arms, reflecting our prior beliefs.
- If the prior is $B(\alpha, \beta)$ then the posterior is $B(\alpha + 1, \beta)$ or $B(\alpha, \beta + 1)$.
- In each round, TS samples a value for each arm from the corresponding Beta distribution, then selects the arm with the highest **sampled value**. The parameters for the chosen arm are then updated.

Beta Distriubtion

The PDF of $B(\alpha, \beta)$ on $x \in [0, 1]$ is $c(\alpha, \beta)x^{\alpha-1}(1-x)^{\beta-1}$

The mean of $B(\alpha, \beta)$ is $\frac{\alpha}{\alpha+\beta}$.



Bayesian Updating

How Bayesian Updating Works:

- Start with a **prior belief**: $\mathrm{B}(\alpha, \beta)$.
- Collect new data:
  - s: Number of successes (e.g., clicks).
  - f: Number of failures (e.g., no clicks).
- Update your belief to get a **posterior distribution**:

$$\text{Posterior} = (\alpha + s, \beta + f)$$

Example:

- Start with B(10, 40) (prior belief).
- New data: 3 clicks ($s = 3$) and 7 no-clicks ($f = 7$).
- Updated belief (posterior): B(10 + 3, 40 + 7) = B(13, 47).

Bayesian Updating

Bayesian Updating Formula:

$$P(\theta|\text{data}) = \frac{P(\text{data}|\theta)P(\theta)}{P(\text{data})}$$

- $P(\theta|\text{data})$: Posterior probability (updated belief about θ).
- $P(\text{data}|\theta)$: Likelihood of observing the data given θ.
- $P(\theta)$: Prior belief about θ.
- $P(\text{data})$: Evidence (normalizing constant).

Setup:

- θ: Unknown probability of success (e.g., CTR).
- Data: s successes, f failures (e.g., clicks and no-clicks).
- Prior: $\theta \sim \text{B}(\alpha, \beta)$.

What is $P(\theta|\text{data})$ in the frequentist world?

Beta Distribution (Bayes) Updates Easily

Bayesian Updating:

$$P(\text{data}|\theta) \propto \theta^s (1-\theta)^f$$

$$P(\theta|\text{data}) \propto \theta^s (1-\theta)^f \cdot \text{Beta}(\alpha, \beta)$$
$$\propto \theta^{\alpha+s-1} (1-\theta)^{\beta+f-1}$$

Result:

- Posterior: $\text{Beta}(\alpha + s, \beta + f)$.

Conclusion:

- Bayesian updating computationally simple and intuitive for Beta Distribution with Bernoulli arms.

TS: Formal Algorithm (Beta-Bernoulli Model)

Algorithm Thompson Sampling with Beta-Bernoulli model

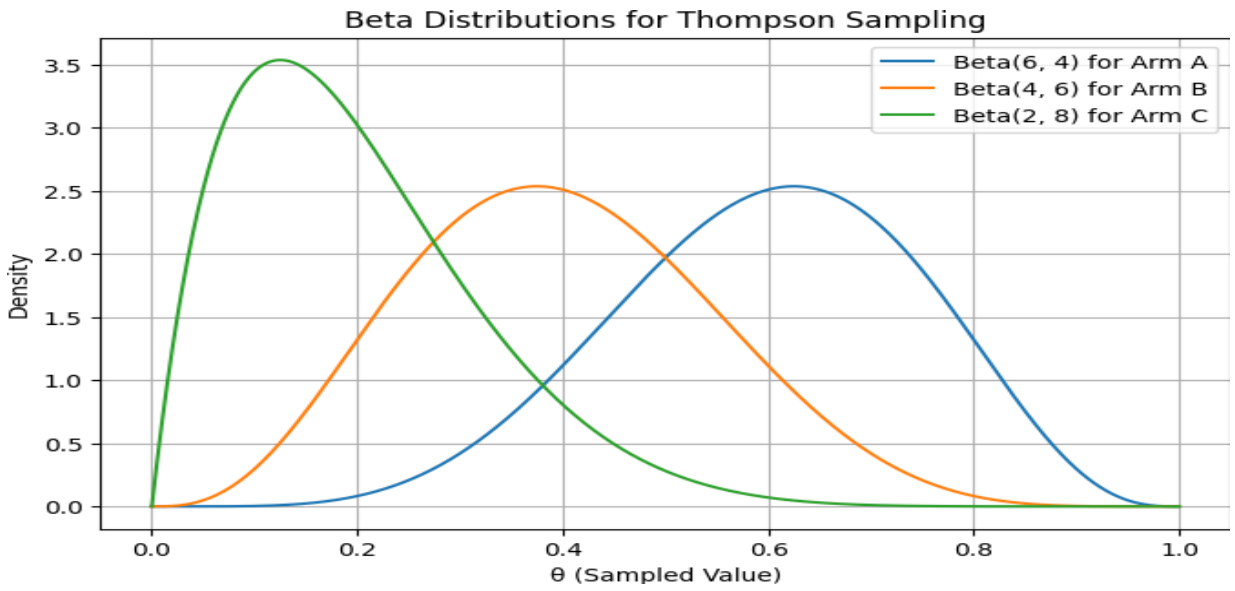
```
Initialize α_a = 1 and β_a = 1 for all arms a.
for each time step t = 1, 2, 3, ... do
    for each arm a do
        Sample θ_a ~ Beta(α_a, β_a).
    end for
    Select the arm a_t with the highest sampled θ_a. Observe reward r_t.
    if r_t = 1 then
        Update α_{a_t} = α_{a_t} + 1.
    else
        Update β_{a_t} = β_{a_t} + 1.
    end if
end for
```

TS Algorithm: Step-by-Step Example

Example: Three arms (A, B, C) at time t.

Arm	Successes (α_a)	Failures (β_a)	Beliefs
A	6	4	$\text{Beta}(6, 4)$
B	4	6	$\text{Beta}(4, 6)$
C	2	8	$\text{Beta}(2, 8)$

Step 1: Each arm samples $\theta_a \sim$ Beta(α_a, β_a)

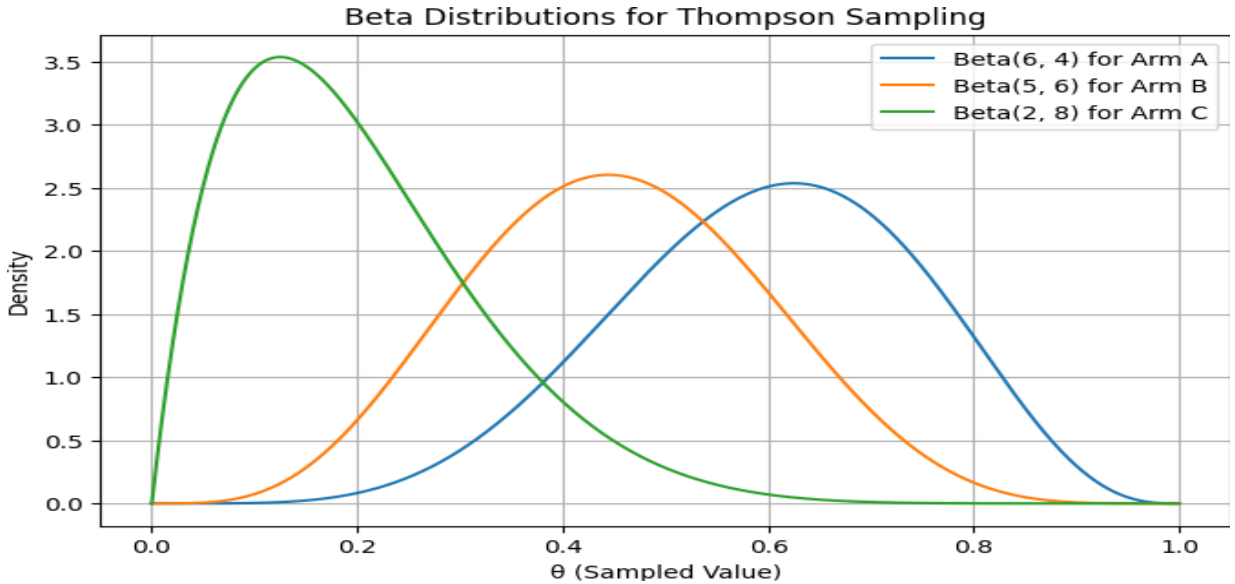


Step 2: Select the Arm with the Highest Sampled θ.

TS Algorithm: Updating Beliefs

Example: Arm B was chosen and resulted in a success ($r_t = 1$).
Step 3: Update the Prior Beliefs: Since Arm B succeeded, we increment $\alpha_B = \alpha_B + 1$.

Arm	**Successes** (α_a)	**Failures** (β_a)	**Beliefs**
A	6	4	Beta$(6, 4)$
B	5	6	Beta$(5, 6)$
C	2	8	Beta$(2, 8)$



Repeat: Steps 1, 2, 3.

MAB in Matching: DoorDash Example

Using a Multi-Armed Bandit to Identify Responsive Dashers

Data Science Problem:

- Matching Dasher supply to consumer demand.
- Old DoorDash approach: Randomly messaging Dashers in a location, which leads to:
  - Low conversion rates.
  - Spamming uninterested Dashers.

Solution: MAB

- Dynamically learns Dasher preferences and optimizes messaging strategy by ranking Dashers based on likelihood to respond.

Assortment Optimization with TS

Problem:

- Same as the UCB problem: 4 restaurants so we have $4! = 24$ assortments.
- Purchase probabilities are unknown, for simplicity, assume random in $[0.3, 0.7]$.
- Goal: Find good assortments with TS.

How would you compare TS to UCB?

Thompson Sampling vs. UCB

- In standard MAB settings, both TS and UCB typically perform well.
- In more complex environments, UCB often requires more fine-tuning to achieve similar performance (confidence bound tuning).
- On the other hand, TS requires sampling from the posterior distribution and Bayesian updating and is computationally expensive.
- UCB is a deterministic algorithm which can be an advantage for some applications.

TS and UCB: Powerful Exploration Tools

- TS (Bayesian) and UCB (Frequentist) are fundamental tools for exploration in online decision-making.
- They balance exploration vs. exploitation efficiently, making them crucial for learning.

Beyond Simple Multi-Armed Bandits:

- Can be extended to more complex problems like **Contextual MABs** and **Reinforcement Learning (RL)**.

Applications Beyond Online Marketplaces:

- **Finance:** Algorithmic trading and portfolio optimization.
- **Cybersecurity:** Adaptive defense strategies against evolving threats.
- Other?

3. Challenges in MAB: Non-Stationary Environments

- UCB and TS work well in static environments where rewards are stationary.
- In many practical scenarios, the enviorment can change (e.g., because the rewards change over time and/or competitors may adapt to exploit the algorithm's strategy).

Non-Stationary Environment: Changing Rewards

- UCB assumes that reward distributions are stationary.
- In real-world applications, rewards can shift over time due to evolving user preferences or changes in the demand.
- Since UCB algorithms rely on cumulative statistics, they may **overweight outdated information**, leading to suboptimal decisions.

What about TS? How would you adjust to non-stationary rewards?

Solution: Restarting / Discounting

Idea: Instead of treating all past data equally, we can **reset or discount older data** to focus on recent trends.

Approaches:

- **Restarting:** Periodically restart UCB or TS by resetting counts and rewards.
- **Sliding Window Approach:** Only the most recent T_w rounds are considered.
- **Discounting Approach:** Discount older rewards.

Benefits:

- Prevents the algorithm from being locked into outdated optimal actions. Responsive to short-term changes.

Discounted UCB

Example: Discounted UCB (D-UCB)

Time t	Arm Pulled A_t	Reward r_t	Discount Factor Applied
1	A_1	0.5	γ^3
2	A_2	0.7	(not used for A_1)
3	A_1	0.2	γ^1

Table: Recent rewards get higher weight than older ones.

Empirical Mean in D-UCB

$$\hat{\mu}_{A_1}^{(4)} = \frac{\gamma^3(0.5) + \gamma^1(0.2)}{\gamma^3 + \gamma^1}$$

- Instead of treating all past rewards equally, Discounted UCB applies **exponential decay**: older rewards **gradually lose influence**.

Sliding Window UCB: Example

Example: Sliding Window UCB with $T_w = 3$.
At time $t = 6$, we use only the most recent $T_w = 3$ rounds (i.e., $t = 3, 4, 5$) to compute the empirical mean.

Time t	**Arm Pulled** A_t	**Reward** r_t
3	A_1	0.2
4	A_1	0.8
5	A_2	0.4

Table: Only the last three rounds ($t = 3, 4, 5$) are used for computing the mean.

Sliding Window Empirical Mean for A_1 at $t = 6$

$$\hat{\mu}_{A_1}^{(6)} = \frac{0.2 \times 1 + 0.8 \times 1 + 0.4 \times 0}{1 + 1 + 0} = \frac{0.2 + 0.8}{2} = 0.5$$

Sliding Window UCB Algorithm

- Play standard UCB until T_w.
  Maintain a sliding window of the last T_w rounds.
- For $t > T_w$, compute empirical mean only over the window:

$$\hat{\mu}_a^{(t)} = \frac{\sum_{s=t-T_w}^{t} r_s \mathbf{1}_{\{I_s=a\}}}{\sum_{s=t-T_w}^{t} \mathbf{1}_{\{I_s=a\}}}$$

- Use standard UCB algorithm within a window:

$$UCB_a(t) = \hat{\mu}_a^{(t)} + \sqrt{\frac{2\log(T_w)}{N_a(T_w)}}$$

where $N_a(T_w)$ is the number of times arm a was played in the last T_w rounds.

Loyalty Program Example

Platforms often use loyalty programs to incentivize users.

In particular, ride-sharing platforms (e.g., Uber, Grab) often use loyalty programs to incentivize drivers to complete more rides.

The Challenge:

- How can we design a program that motivates drivers without overspending?
- How do we measure the success of different loyalty programs?
- How can we adapt the program as driver behavior evolves?

Example: Two Loyalty Program Options

We are considering two possible **daily** bonus structures:

- Program A:
  - \$5 bonus after completing 5 rides in a day.
  - \$10 bonus after completing 8 rides in a day.
- Program B:
  - \$4 bonus after completing 5 rides in a day.
  - \$11 bonus after completing 8 rides in a day.

Question: Which program will better motivate drivers to complete more rides?

Would you try an A/B test ?

If you run MAB algorithm (say UCB) What metric should define success? what is an iteration t?

Possible Objective

The daily completion rates:

$$\text{Completion Rate (5 rides)} = \frac{\text{Number of drivers completing at least 5 rides}}{\text{Number of active drivers}}$$

$$\text{Completion Rate (8 rides)} = \frac{\text{Number of drivers completing at least 8 rides}}{\text{Number of active drivers}}$$

Objective: Average / weighted completion rates.

Time Structure

Incentives are defined per day, so tracking performance daily makes sense.

So for each day (not each driver) we choose Program A or Program B.

But driver behavior evolves over time, older data may become irrelevant...

Sliding Window:

- Focuses on the most recent T_w days (e.g., last 20 days).
- Adapts quickly to new trends (e.g., increased demand).

Sliding Window Example

Setup:

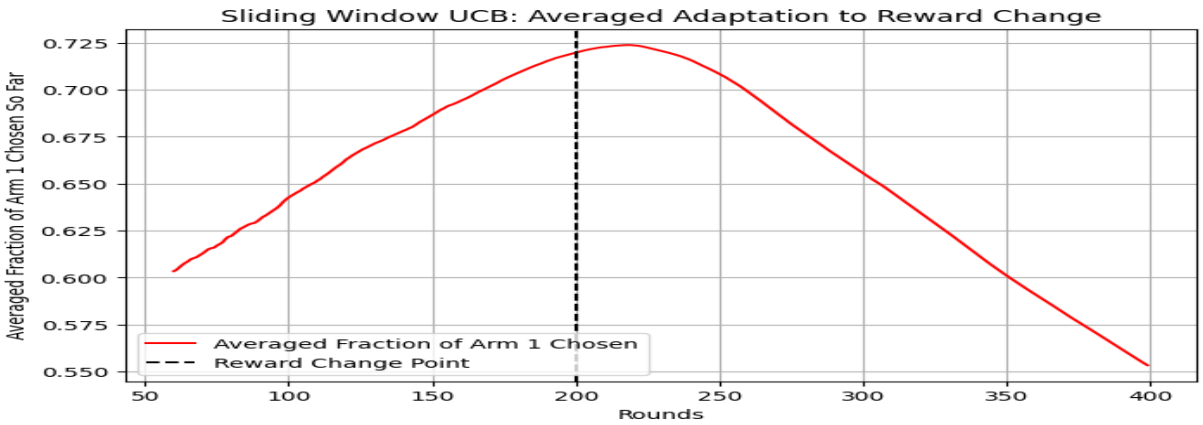
- Two arms: Arm 0 (reward starts at 0.45) and Arm 1 (reward starts at 0.55).
- Reward for Arm 1 changes to 0.45 and Arm 0 to 0.55 at time $T/2$. Sliding Window T_w.

Implementation Details:

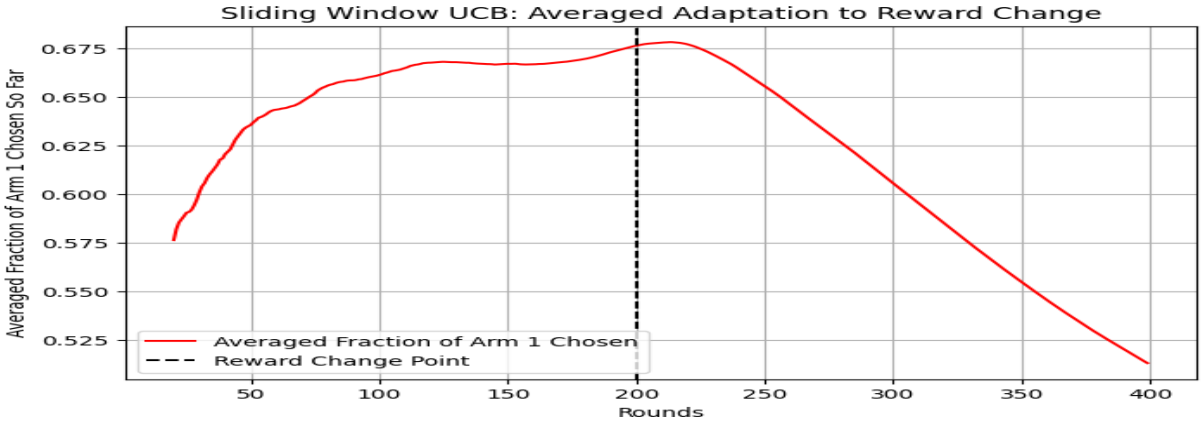
- First T_w rounds:
  - Play standard UCB.
- From T_w: UCB Selection with Sliding Window:
  - At each round, compute empirical means using only the last T_w rounds.
  - If an arm was not played in the window, default to 0.5.
  - Play according to sliding window UCB algorithm.

Sliding Window UCB Results

$T = 400,\ T_w = 60.$ Results averaged over 200 simulations.



$T = 400,\ T_w = 20.$ What is the trade-off?



Short Horizon: Hawker Center Example

- Imagine you live next to a hawker center with many laksa stalls.
- Your goal is to find the shiokest laksa.
- Would you use epsilon-greedy / UCB / TS ?

Got Problem:

- First 10 visits: your choices are mainly random: can try suboptimal laksa most times.
- Maybe you'd do something more kiasu: Check reviews, ask friends, or follow the longest queue. (Long queue, better food, right?)

Key Point:

- Algorithms like / UCB / TS are designed for **long-term** horizon problems when you have enough time (many trials) to explore and exploit effectively.

Challenges with Short Horizons

What If Time Is Limited?

- TS / UCB algorithms spend too much time exploring for problems like the hawker center example.
- In short horizons, the early randomness can lead to poor decisions overall.
- How can we still perform well?

Strategies to Improve Performance in Short Horizons

Solutions:

- **Strong Prior:**
  - Use historical data, reviews, or expert knowledge to set good starting beliefs (priors).
- **Discounting:**
  - Optimize a discounted objective.
- **Causal Inference:**
  - Identify the true effect of an action.
- In short horizons, leveraging strong priors and causal insights can complement bandit algorithms to make better decisions.

Competitive Settings

Example: Pricing in marketplaces where competitors react to price changes.

		Player 2	
		p_l	p_h
Player 1	p_l	$1;1$	$15;2$
	p_h	$2;15$	$7;7$

Table: If Player 2 knows Player 1 uses UCB or TS, they can always play p_l, forcing Player 1 to settle for p_h, which benefits Player 2.

Takeaway:

- Algorithms like UCB or TS may perform poorly when rewards depend on the actions of competitors.
- There are algorithms that they to address this (how? more randomization is key)

Contextual MAB

- Builds on the standard MAB by incorporating **contextual information** available at each decision point.
- At each time t, we observe a **context vector** x_t, which represents relevant information about the environment.
- Rewards depend on both:
  - The action (arm) chosen.
  - The observed context x_t.

Examples in Marketplaces?

Contextual MAB in Online Marketplaces

- **Recommendation Systems:**
  - Context x_t: User features (e.g., demographics, location, browsing history).
  - Arms: Items to recommend (e.g., movies, products, or songs).
  - Reward: User engagement (e.g., clicks, purchases, or watch time).
- **Dynamic Pricing:**
  - Context x_t: ?
  - Arms: Different prices.
  - Reward: Revenue from chosen price (or other objective).
- **Supply Allocation in Delivery Platforms:**
  - Context x_t: Order features (e.g., delivery location, item size, estimated preparation time).
  - Arms: Delivery partners (e.g., Dashers).
  - Reward: ?
  - Example: GrabFood assigns delivery orders to drivers.

Contextual Bandits: Personalization and Adaptation

How Does it Work?

- Algorithms like **UCB** and **TS** can be adapted to contextual settings by incorporating context vectors into the decision-making process.

LinUCB: A Contextual Bandit Algorithm

- A natural extension of UCB for **contextual bandits**.
- Assumes that the reward is a **linear function of the context**.

LinUCB:

- At each time t, observe a **context vector** $x_i(t)$ for each arm i.
- Estimate the expected reward for each arm as:

$$\hat{r}_i(t) = x_i(t)^\top \hat{\theta}_i(t)$$

- $\hat{\theta}_i(t)$: Estimated parameter vector for arm i (learned up to time t).
- Add a bonus (what does the bonus quantify)?

$$UCB_i(t) = x_i(t)^\top \hat{\theta}_i(t) + \text{bonus factor}_i$$

- Learns the relationship between context and reward over time.

Contextual Thompson Sampling: Big Picture

TS for contextual MAB:

1. Assume a **linear reward model**.
2. Maintain a **belief** over θ_i.
3. Sample: Draw $\tilde{\theta}_i(t)$ from the posterior for each arm i.
4. Estimate reward for arm i:

$$\hat{r}_i(t) = x_i(t)^\top \tilde{\theta}_i(t).$$

5. Choose the arm with the highest predicted reward:

$$a_t = \arg\max_i \hat{r}_i(t).$$

6. Update belief using Bayesian updating.

Practical Considerations: Building the Context Vector

Understand the Problem Domain

- Identify the key factors/ metrics that influence decisions.

Extract Contextual Features

- Features should capture information relevant to decision-making.
- Example:
  - **Recommendation Systems:** User demographics, browsing history, product attributes.
- Ensure that all features are available in real-time for decision-making.

Practical Considerations: Modeling the Context Vector

Dimensionality Reduction

- Large feature vectors can make computation inefficient.
- Use techniques like:
  - **Principal Component Analysis (PCA):** Combine features into fewer components.
  - **Feature Selection:** Use domain expertise to select the most relevant features.
  - Neural Networks?

Use Classifiers

- If the context vector includes categorical data (e.g., user demographics), encode it efficiently.

Neural Networks in Bandits: Bidding Example

Example: Bidding in Online Advertising

- Context vectors for users or agents can be huge:
  - User browsing history, demographics, and behavioral data.
  - Competitor bids, market trends, and other dynamic factors.
- **Problem:** Directly using these high-dimensional vectors is too slow for real-time bidding.
- **Solution:** Use a neural network offline to summarize these features into a single value:
  - Example: Predict the **likelihood of buying** or **estimated value of a user**.
  - Pass this summarized value as the context for the bandit algorithm.
- Enables the bandit to operate online efficiently while leveraging rich offline data.

Other examples?

Data Science Team Experimentation

Data Science teams have many steps to make it "work". e.g.,

1. **Define the Problem and Goals**
   - Clearly articulate goals. Translate it into measurable metrics.
2. **Collect and Prepare Deployment**
   - Gather historical data on contexts, actions, and rewards.
   - Engineer "good" features, reduce dimensions, etc.
3. **Deploy and Iterate**
   - Deploy algorithm in a controlled environment.
   - Monitor performance and refine features / actions. Scale gradually.

This requires effective collaboration between business, data science, and engineering teams!!

Discussion: Experimentation and AI

1. **Experimentation in the Age of AI**
   - Can AI fully replace A/B testing and MAB or does it serve a complementary role?
2. **AI and Data Science: Complementary or Substitutable?**
   - Will AI eventually replace data scientists, or will it primarily enhance their capabilities?

Discussion: Algorithmic Collusion

if decentralized algorithms lead to monopoly pricing, do you think regulators should intervene? possible intervention methods?

What are arguments against regulatory intervention (what are challenges associated with intervention)?